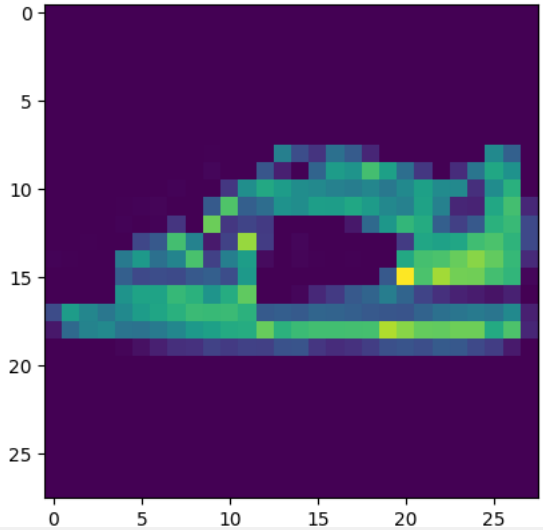
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import keras
from keras.datasets import fashion_mnist


```

```
(train_x ,train_y),(test_x,test_y)=fashion_mnist.load_data() #train test images and lables

```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
plt.imshow(train_x[13])

```

```
<matplotlib.image.AxesImage at 0x7839899b4910>
```




```
train_y[13]

```

```
5
```

```
print(train_x.shape)
print(train_y.shape)
print(test_x.shape)
print(test_y.shape)
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

```
#pre processing
#CNN starts
#1 Convert 2D -> 3D

train_x=train_x.reshape(-1,28,28,1)
test_x=test_x.reshape(-1,28,28,1)
```

```
#min max scaler
train_x=train_x.astype(np.float32)
test_x=test_x.astype(np.float32)
```

```
train_x=train_x/255
test_x=test_x/255
```

```
from  keras.utils import to_categorical
train_y_one_hot=to_categorical(train_y)
test_y_one_hot=to_categorical(test_y)
```

```
print(train_y[10])
print(train_y_one_hot[10])
```

```
0
[1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
#model Buliding
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout,Activation,Flatten,Conv2D
```

```
from sklearn.model_selection import train_test_split
train_x, valid_x, train_lables, valid_lables = train_test_split(train_x, train_y_one_hot, test_size=0.20)

```

```
#model Desgin
from keras.layers import MaxPooling2D,LeakyReLU
```

```
model=tf.keras.Sequential()

#Hidden Layer 1
model.add(Conv2D(32,(3,3),activation='linear',input_shape=(28,28,1),
padding='same'))
model.add(LeakyReLU(alpha=0.1))
model.add(MaxPooling2D((2,2),padding='same'))
model.add(Dropout(0.25))
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.10/dist-packages/keras/src/layers/activations/leaky_relu.py:41: UserWarning: Argument `alpha` is deprecated.
  warnings.warn(
```

```
#Hidden Layer 2

model.add(Conv2D(64,(3,3),activation='linear',
padding='same'))
model.add(LeakyReLU(alpha=0.1))
model.add(MaxPooling2D((2,2),padding='same'))
model.add(Dropout(0.25))
```

```
#Hidden Layer 3

model.add(Conv2D(128,(3,3),activation='linear',
padding='same'))
model.add(LeakyReLU(alpha=0.1))
model.add(MaxPooling2D((2,2),padding='same'))
model.add(Dropout(0.25))
```

```
model.add(Flatten())
```

```
#OUTPUT Layer
model.add(Dense(128,activation='linear'))
model.add(LeakyReLU(alpha=0.1))
model.add(Dense(10,activation='softmax'))
```

```
model.compile(loss='categorical_crossentropy',optimizer='adam',metrics=['accuracy'])
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
leaky_re_lu (LeakyReLU)	(None, 28, 28, 32)	0
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
dropout (Dropout)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_1 (Dropout)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73,856
leaky_re_lu_2 (LeakyReLU)	(None, 7, 7, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
leaky_re_lu_3 (LeakyReLU)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

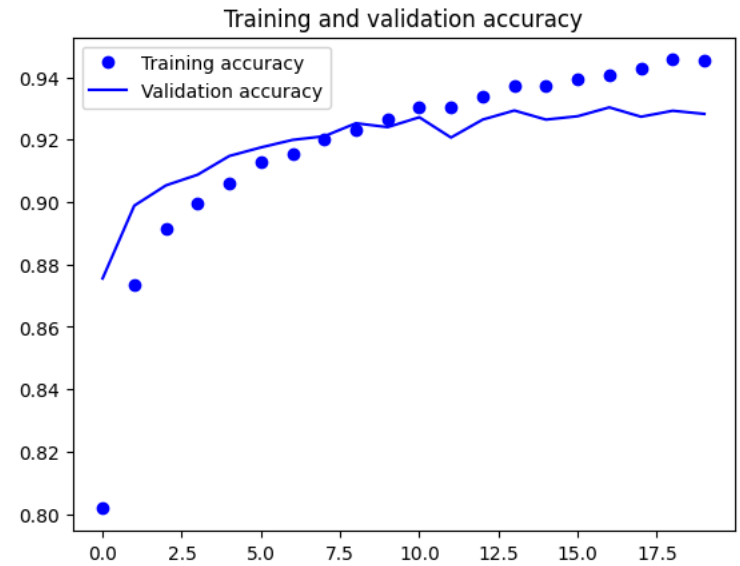
```
model_train=model.fit(train_x,train_lables,batch_size=64,epochs=20,validation_data=(valid_x,valid_lables))
```

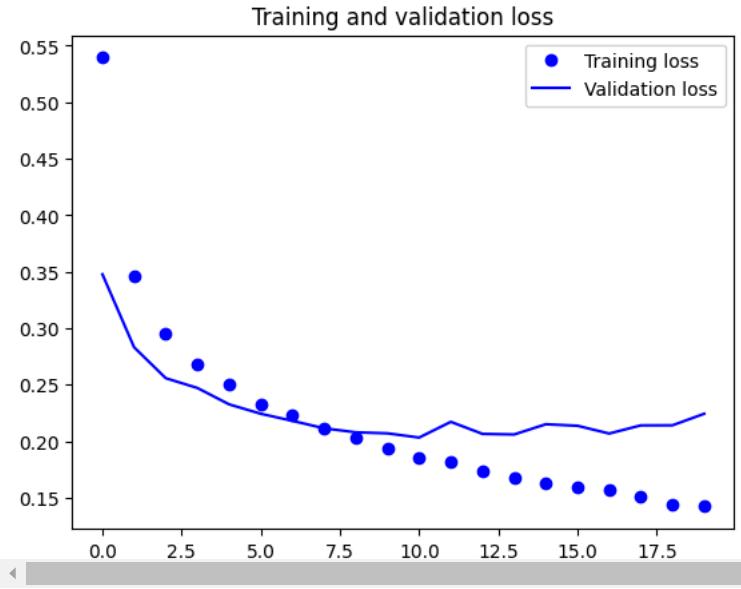
```
Epoch 1/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 107s 139ms/step - accuracy: 0.7115 - loss: 0.7741 - val_accuracy: 0.8755 - val_loss: 0.3477
Epoch 2/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 138s 134ms/step - accuracy: 0.8674 - loss: 0.3636 - val_accuracy: 0.8988 - val_loss: 0.2831
Epoch 3/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 99s 132ms/step - accuracy: 0.8876 - loss: 0.3020 - val_accuracy: 0.9053 - val_loss: 0.2558
Epoch 4/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 99s 132ms/step - accuracy: 0.8975 - loss: 0.2712 - val_accuracy: 0.9087 - val_loss: 0.2471
Epoch 5/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 141s 130ms/step - accuracy: 0.9067 - loss: 0.2494 - val_accuracy: 0.9147 - val_loss: 0.2326
Epoch 6/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 143s 132ms/step - accuracy: 0.9127 - loss: 0.2328 - val_accuracy: 0.9175 - val_loss: 0.2243
Epoch 7/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 99s 131ms/step - accuracy: 0.9179 - loss: 0.2173 - val_accuracy: 0.9199 - val_loss: 0.2180
Epoch 8/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 143s 133ms/step - accuracy: 0.9203 - loss: 0.2086 - val_accuracy: 0.9211 - val_loss: 0.2114
Epoch 9/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 141s 133ms/step - accuracy: 0.9223 - loss: 0.2046 - val_accuracy: 0.9252 - val_loss: 0.2079
Epoch 10/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 142s 132ms/step - accuracy: 0.9297 - loss: 0.1901 - val_accuracy: 0.9240 - val_loss: 0.2070
Epoch 11/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 143s 133ms/step - accuracy: 0.9318 - loss: 0.1805 - val_accuracy: 0.9272 - val_loss: 0.2032
Epoch 12/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 100s 133ms/step - accuracy: 0.9312 - loss: 0.1785 - val_accuracy: 0.9207 - val_loss: 0.2172
Epoch 13/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 100s 133ms/step - accuracy: 0.9362 - loss: 0.1715 - val_accuracy: 0.9264 - val_loss: 0.2065
Epoch 14/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 144s 135ms/step - accuracy: 0.9382 - loss: 0.1638 - val_accuracy: 0.9293 - val_loss: 0.2060
Epoch 15/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 142s 136ms/step - accuracy: 0.9377 - loss: 0.1622 - val_accuracy: 0.9264 - val_loss: 0.2150
Epoch 16/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 145s 139ms/step - accuracy: 0.9403 - loss: 0.1571 - val_accuracy: 0.9275 - val_loss: 0.2137
Epoch 17/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 137s 134ms/step - accuracy: 0.9419 - loss: 0.1547 - val_accuracy: 0.9303 - val_loss: 0.2068
Epoch 18/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 100s 134ms/step - accuracy: 0.9434 - loss: 0.1471 - val_accuracy: 0.9273 - val_loss: 0.2140
Epoch 19/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 101s 134ms/step - accuracy: 0.9482 - loss: 0.1382 - val_accuracy: 0.9293 - val_loss: 0.2141
Epoch 20/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 100s 134ms/step - accuracy: 0.9471 - loss: 0.1368 - val_accuracy: 0.9283 - val_loss: 0.2243
```

```
test_eval = model.evaluate(test_x, test_y_one_hot, verbose=0)
print('Test loss:', test_eval[0])
print('Test accuracy:', test_eval[1])
```

```
Test loss: 0.2476484179496765
Test accuracy: 0.9196000099182129
```

```
accuracy = model_train.history['accuracy']
val_accuracy = model_train.history['val_accuracy']
loss = model_train.history['loss']
val_loss = model_train.history['val_loss']
epochs = range(len(accuracy))
plt.plot(epochs, accuracy, 'bo', label='Training accuracy')
plt.plot(epochs, val_accuracy, 'b', label='Validation accuracy')
plt.title('Training and validation accuracy')
plt.legend()
plt.figure()
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```






```
num_classes = 10
fashion_model = tf.keras.Sequential()
fashion_model.add(tf.keras.layers.Conv2D(32, kernel_size=(3, 3),activation='linear',padding='same',input_shape=(28,28,1)))
fashion_model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
fashion_model.add(tf.keras.layers.MaxPooling2D((2, 2),padding='same'))
fashion_model.add(tf.keras.layers.Dropout(0.25))
fashion_model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='linear',padding='same'))
fashion_model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
fashion_model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2),padding='same'))
fashion_model.add(tf.keras.layers.Dropout(0.25))
fashion_model.add(tf.keras.layers.Conv2D(128, (3, 3), activation='linear',padding='same'))
fashion_model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
fashion_model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2),padding='same'))
fashion_model.add(tf.keras.layers.Dropout(0.4))
fashion_model.add(tf.keras.layers.Flatten())
fashion_model.add(tf.keras.layers.Dense(128, activation='linear'))
fashion_model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
```

```
fashion_model.add(tf.keras.layers.Dropout(0.3))
fashion_model.add(tf.keras.layers.Dense(num_classes, activation='softmax'))
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.10/dist-packages/keras/src/layers/activations/leaky_relu.py:41: UserWarning: Argument `alpha` is deprecated.
  warnings.warn(
```

```
fashion_model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 28, 28, 32)	320
leaky_re_lu_8 (LeakyReLU)	(None, 28, 28, 32)	0
max_pooling2d_6 (MaxPooling2D)	(None, 14, 14, 32)	0
dropout_7 (Dropout)	(None, 14, 14, 32)	0
conv2d_7 (Conv2D)	(None, 14, 14, 64)	18,496
leaky_re_lu_9 (LeakyReLU)	(None, 14, 14, 64)	0
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_8 (Dropout)	(None, 7, 7, 64)	0
conv2d_8 (Conv2D)	(None, 7, 7, 128)	73,856
leaky_re_lu_10 (LeakyReLU)	(None, 7, 7, 128)	0
max_pooling2d_8 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_9 (Dropout)	(None, 4, 4, 128)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_3 (Dense)	(None, 128)	262,272
leaky_re_lu_11 (LeakyReLU)	(None, 128)	0
dropout_10 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 10)	1,290

```
fashion_model.compile(loss=tf.keras.losses.categorical_crossentropy, optimizer=tf.keras.optimizers.Adam(),metrics=['accuracy'])
```

```
fashion_train_dropout = fashion_model.fit(train_x, train_lables, batch_size=64,epochs=20,verbose=1,validation_data=(valid_x, valid_la
```

```
Epoch 1/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 129s 167ms/step - accuracy: 0.6875 - loss: 0.8474 - val_accuracy: 0.8723 - val_loss: 0.3566
Epoch 2/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 116s 132ms/step - accuracy: 0.8582 - loss: 0.3878 - val_accuracy: 0.8930 - val_loss: 0.2943
Epoch 3/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 145s 137ms/step - accuracy: 0.8753 - loss: 0.3372 - val_accuracy: 0.8986 - val_loss: 0.2723
Epoch 4/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 138s 131ms/step - accuracy: 0.8869 - loss: 0.3064 - val_accuracy: 0.9063 - val_loss: 0.2510
Epoch 5/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 143s 132ms/step - accuracy: 0.8961 - loss: 0.2817 - val_accuracy: 0.9127 - val_loss: 0.2392
Epoch 6/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 141s 131ms/step - accuracy: 0.9009 - loss: 0.2698 - val_accuracy: 0.9091 - val_loss: 0.2379
Epoch 7/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 99s 132ms/step - accuracy: 0.9039 - loss: 0.2534 - val_accuracy: 0.9191 - val_loss: 0.2202
Epoch 8/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 141s 130ms/step - accuracy: 0.9070 - loss: 0.2504 - val_accuracy: 0.9182 - val_loss: 0.2256
Epoch 9/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 143s 131ms/step - accuracy: 0.9113 - loss: 0.2387 - val_accuracy: 0.9176 - val_loss: 0.2273
Epoch 10/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 98s 131ms/step - accuracy: 0.9136 - loss: 0.2279 - val_accuracy: 0.9261 - val_loss: 0.2079
Epoch 11/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 142s 131ms/step - accuracy: 0.9149 - loss: 0.2256 - val_accuracy: 0.9233 - val_loss: 0.2122
Epoch 12/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 98s 131ms/step - accuracy: 0.9164 - loss: 0.2204 - val_accuracy: 0.9201 - val_loss: 0.2198
Epoch 13/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 146s 137ms/step - accuracy: 0.9182 - loss: 0.2133 - val_accuracy: 0.9283 - val_loss: 0.2079
Epoch 14/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 99s 132ms/step - accuracy: 0.9191 - loss: 0.2193 - val_accuracy: 0.9247 - val_loss: 0.2056
Epoch 15/20
```

```
750/750 ━━━━━━━━━━━━━━━━━━━━ 141s 131ms/step - accuracy: 0.9220 - loss: 0.2113 - val_accuracy: 0.9264 - val_loss: 0.2054
Epoch 16/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 141s 130ms/step - accuracy: 0.9236 - loss: 0.2032 - val_accuracy: 0.9272 - val_loss: 0.2049
Epoch 17/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 142s 131ms/step - accuracy: 0.9215 - loss: 0.2090 - val_accuracy: 0.9254 - val_loss: 0.2114
Epoch 18/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 142s 131ms/step - accuracy: 0.9229 - loss: 0.2010 - val_accuracy: 0.9278 - val_loss: 0.2100
Epoch 19/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 98s 131ms/step - accuracy: 0.9251 - loss: 0.1978 - val_accuracy: 0.9239 - val_loss: 0.2186
Epoch 20/20
750/750 ━━━━━━━━━━━━━━━━━━━━ 98s 131ms/step - accuracy: 0.9256 - loss: 0.1962 - val_accuracy: 0.9271 - val_loss: 0.2039
```

```
fashion_model.save("fashion_model_dropout.keras")

```

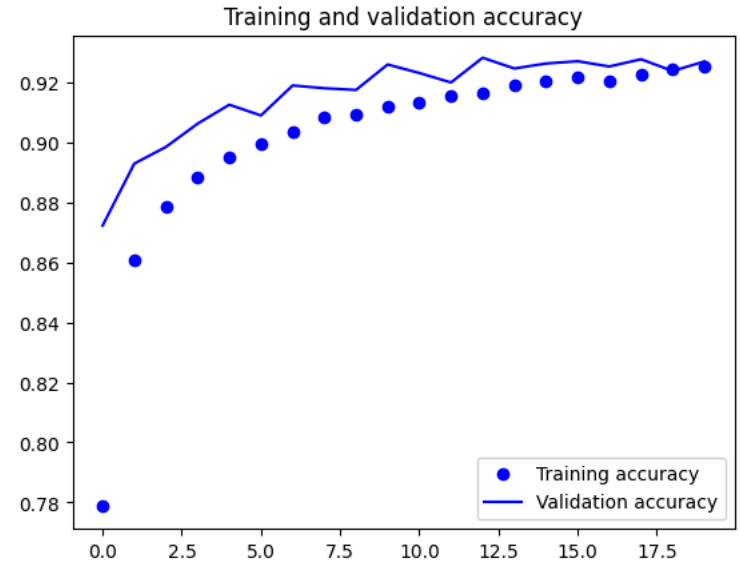
```
test_eval = fashion_model.evaluate(test_x, test_y_one_hot, verbose=1)

```

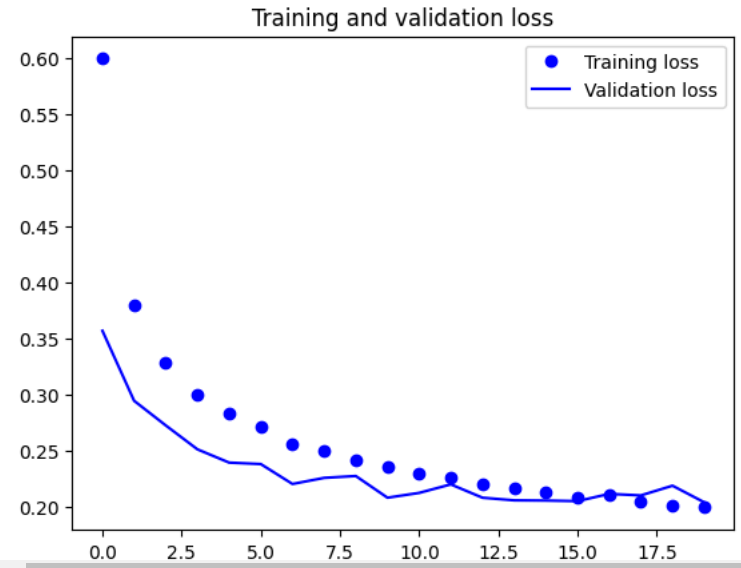
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9223 - loss: 0.2161
```

```
print('Test loss:', test_eval[0])
print('Test accuracy:', test_eval[1])
```

```
Test loss: 0.21556106209754944
Test accuracy: 0.9226999878883362
```

```
accuracy = fashion_train_dropout.history['accuracy']
val_accuracy = fashion_train_dropout.history['val_accuracy']
loss = fashion_train_dropout.history['loss']
val_loss = fashion_train_dropout.history['val_loss']
epochs = range(len(accuracy))
plt.plot(epochs, accuracy, 'bo', label='Training accuracy')
plt.plot(epochs, val_accuracy, 'b', label='Validation accuracy')
plt.title('Training and validation accuracy')
plt.legend()
plt.figure()
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```





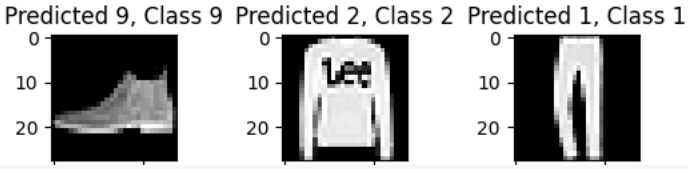

```
predicted_classes = fashion_model.predict(test_x)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step
```

```
predicted_classes = np.argmax(np.round(predicted_classes),axis=1)

```

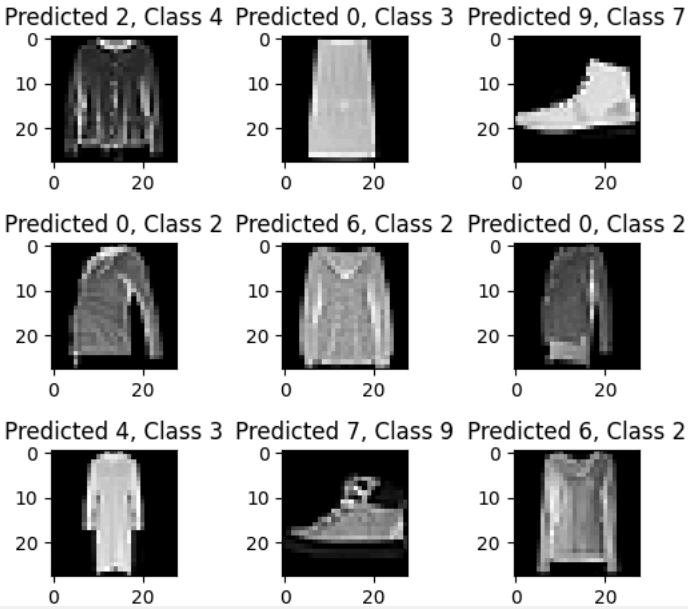
```
correct = np.where(predicted_classes==test_y)[0]
print ("Found %d correct labels" % len(correct))
for i, correct in enumerate(correct[:9]):
    plt.subplot(3,3,i+1)
    plt.imshow(test_x[correct].reshape(28,28), cmap='gray', interpolation='none')
    plt.title("Predicted {}, Class {}".format(predicted_classes[correct], test_y[correct]))
    plt.tight_layout()
```

```
Found 9193 correct labels
```




```
incorrect = np.where(predicted_classes!=test_y)[0]
print ("Found %d incorrect labels" % len(incorrect))
for i, incorrect in enumerate(incorrect[:9]):
    plt.subplot(3,3,i+1)
    plt.imshow(test_x[incorrect].reshape(28,28), cmap='gray', interpolation='none')
    plt.title("Predicted {}, Class {}".format(predicted_classes[incorrect], test_y[incorrect]))
    plt.tight_layout()
```

```
Found 807 incorrect labels
```




```
from sklearn.metrics import classification_report
target_names = ["Class {}".format(i) for i in range(num_classes)]
print(classification_report(test_y, predicted_classes, target_names=target_names))
```

```
              precision    recall  f1-score   support

     Class 0       0.79      0.89      0.84      1000
     Class 1       1.00      0.99      0.99      1000
     Class 2       0.92      0.84      0.88      1000
     Class 3       0.94      0.92      0.93      1000
     Class 4       0.84      0.91      0.88      1000
     Class 5       0.99      0.98      0.99      1000
     Class 6       0.81      0.74      0.78      1000
     Class 7       0.97      0.96      0.97      1000
     Class 8       0.99      0.98      0.99      1000
     Class 9       0.96      0.98      0.97      1000

    accuracy                           0.92     10000
   macro avg       0.92      0.92      0.92     10000
weighted avg       0.92      0.92      0.92     10000
```